14.5 Upper and Lower Bounds in Generics

👉Introduction

Upper and lower bounds in generics extend the concept of wildcards in Java. They allow us to increase or decrease the restrictions on type variables, providing flexibility when working with generic types.

👉Upper Bounded Wildcards

What are Upper Bounded Wildcards?

Upper bounded wildcards decrease the restrictions on a variable by allowing the unknown type to be a specific type or any of its subtypes. This is done by using the wildcard character (?) followed by the extends keyword, which indicates that the wildcard type must be a subclass (or implementer) of a specified class or interface.

Syntax

List<? extends Number>

Explanation:

- ? is a wildcard character.
- extends is a keyword.
- Number is a class from the java.lang package.

Using List<? extends Number> allows for a list of Number or any of its subclasses (like Integer or Double). This is more flexible than using List<Number>, which only allows for a list of type Number.

👉Example: Upper Bounded Wildcards

```
class Human {
    public void sleep() {
        System.out.println("Humans need to sleep well...");
    }
}

class Employee extends Human {
    public void sleep() {
        System.out.println("Employees need to sleep well to
stay productive...");
    }
```

```
}

public class Demo {
    public static void main(String[] args) {
        Human human = new Human();
        Employee emp = new Employee();

        ArrayList<?> humanList = new ArrayList<>();
        ArrayList<?> empList = new ArrayList<>();

        humanList = empList; // This assignment is valid.

        // Upper bound wildcard
        ArrayList<? extends Human> humanList2 = new
ArrayList<>();

        /*
        The wildcard is bounded with the upper limit
restriction of the Human class and its subclasses.
        This means the reference can only be of type Human or
its extending classes.
        */

        humanList2 = empList; // Valid, as Employee extends
Human.

        humanList2 = humanList; // Also valid, as Human itself
is allowed.

        // String as Wildcard
        ArrayList<String> stringList = new ArrayList<>();

        // humanList = stringList; // Invalid, String and
Human are not related.
    }
}
```

👉Explanation

In the above example, the upper bounded wildcard allows references of type Human and its subclasses. However, unrelated types like String cannot be used.

👉Lower Bounded Wildcards

What are Lower Bounded Wildcards?

Lower bounded wildcards restrict the unknown type to be a specific type or a supertype of that type. This is achieved by using the wildcard character (?) followed by the super keyword.

Syntax

List<? super Integer>

Explanation:

- ? is a wildcard character.
- super is a keyword.
- Integer is a wrapper class.

Using List<? super Integer> allows for a list of Integer or any of its superclasses (Number or Object). This is more flexible than List<Integer>, which only allows for lists of type Integer.

👉Example: Lower Bounded Wildcards

```
class Human {
    public void sleep() {
        System.out.println("Humans need to sleep well...");
    }
}

class Employee extends Human {
    public void sleep() {
        System.out.println("Employees need to sleep well to
stay productive...");
    }
}

public class Demo {
    public static void main(String[] args) {
        Human human = new Human();
        Employee emp = new Employee();

        ArrayList<? super Human> humanList = new
ArrayList<>();
        ArrayList<Employee> empList = new ArrayList<>();
```

```
        // humanList = empList; // Invalid, Employee is a
subclass, not a superclass of Human.

        ArrayList<Object> objList = new ArrayList<>();

        humanList = objList; // Valid, as Object is a
superclass of Human.

        // ArrayList<String> stringList = new ArrayList<>();
        // humanList = stringList; // Invalid, String is not
related to Human.
    }
}
```

👉Explanation

In this example, the lower bounded wildcard only allows types that are Human or its superclasses, such as Object. Assigning a list of Employee to humanList is not allowed since Employee is a subclass.

👉Upper and Lower Bound Example with Methods

Example 1: Using Upper Bound in a Method

```
public class Demo {
    public static void invokeSleep(List<? extends Human> list)
{
        for (Human human : list) {
            human.sleep();
        }
    }

    public static void main(String[] args) {
        Human human1 = new Human();
        Employee emp1 = new Employee();

        ArrayList<Human> humanList = new ArrayList<>();
        ArrayList<Employee> empList = new ArrayList<>();

        humanList.add(human1);
        empList.add(emp1);

        // Invoking with a list of Employee objects
        invokeSleep(empList); // Allowed, as Employee extends
```

```
Human.

        // Invoking with a list of Human objects
        invokeSleep(humanList); // Also allowed.
    }
}
```

👉Explanation:

The *invokeSleep()* method accepts a List<? extends Human>, meaning it can take lists of Human or any subclass. This allows both humanList and empList to be passed to the method.

👉Example 2: Modifying invokeSleep Method with Lower Bound

```
public class Demo {
    public static void invokeSleep(List<? super Human> list) {
        for (Object obj : list) {
            // Cannot assume 'obj' is a Human, so casting
would be needed to call Human-specific methods.
        }
    }

    public static void main(String[] args) {
        ArrayList<Object> objList = new ArrayList<>();
        objList.add(new Object());

        invokeSleep(objList); // Valid, as Object is a
superclass of Human.

        ArrayList<Employee> empList = new ArrayList<>();
        // invokeSleep(empList); // Invalid, Employee is not a
superclass of Human.
    }
}
```

👉Explanation:

With lower bounds, only lists of Human or its superclasses (Object) are allowed. The method cannot accept lists of subclasses like Employee.

👉Conclusion

Upper and lower bounded wildcards in Java provide flexibility in handling generic types.

- **Upper bounded wildcards (? extends T)** allow a generic type to accept a type and its subclasses.
- **Lower bounded wildcards (? super T)** allow a generic type to accept a type and its superclasses.